Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen VIII: Desarrollo Web, Formatos de Datos, Bases de Datos y Frameworks

Enciclopedia de Programación e Ingeniería del Software — Partes VI, VII y VIII

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Partes VI−VIII: Web, Almacenamiento y Ecosistemas de Producción 1

1. HTML Living Standard y Accesibilidad Web (WCAG / ARIA) 2
1.1. Naturaleza declarativa de HTML y el estándar vivo WHATWG y Semántica estructural, árbol de accesibilidad (AOM) y WAI-ARIA . . . 2
1.2. Formularios nativos, validación de restricciones y seguridad en enlaces y Carga eficiente de recursos, módulos ES y Web Components . . . 3

2. CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries 5
2.1. Algoritmo de la Cascada, Capas (@layer) y Especificidad Lexicográfica y Modelo de Caja, Box-Sizing y Propiedades Lógicas Internacionalizadas . . . 5
2.2. Contextos de Formato: Flexbox Unidimensional y CSS Grid Bidimensional y Container Queries, Rendimiento de Animaciones y Compositor GPU . . . 6

3. Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf 8
3.1. JSON (RFC 8259) y Validación Contractual con JSON Schema y YAML 1.2 vs TOML v1.0: Configuración Humana y Trampas Semánticas . . . 8
3.2. XML, Namespaces, XSD y Prevención de Ataques XXE y CSV (RFC 4180), Markdown (CommonMark) y Serialización Binaria con Protocol Buffers . . . 9

4. Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF) 11
4.1. El Modelo Relacional de Codd y las Claves Relacionales y Dependencias Funcionales, Axiomas de Armstrong y Cierre de Atributos . . . 11
4.2. Anomalías de Diseño y Jerarquía de Formas Normales (1NF, 2NF, 3NF y BCNF) y Descomposición sin Pérdida (Lossless-Join) y Preservación de Dependencias . . . 11

5. Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC 14
5.1. Árboles B+ (B+-Trees) frente a LSM-Trees (Log-Structured Merge-Trees) y Diseño de Índices Compuestos, Índices Cubrientes (Covering) y Optimizadores de Costes . . . 14
5.2. Transacciones ACID, Write-Ahead Logging (WAL) y Protocolo ARIES y Niveles de Aislamiento, Anomalías de Concurrencia y MVCC (Snapshot Isolation) . . . 15

6. Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis 17
6.1. Teorema CAP y Clasificación PACELC en Almacenamiento Distribuido y Replicación Líder-Seguidor, Quórums de Lectura/Escritura y Sharding . . . 17
6.2. PostgreSQL y SQLite en Producción: De Sistemas Embebidos a Escala Empresarial y Motores NoSQL Especializados: Documentos (MongoDB) y Memoria Clave-Valor (Redis) 17

7. Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js 20
7.1. Del DOM Imperativo a la UI Declarativa: Función Estado -> Vista y React y Next.js: Reconciliación Fiber, Hooks, Server Components e Hidratación . . . 20
7.2. Angular, Vue y Svelte: Inyección de Dependencias, Proxies Reactivos y Compilación y Gestión de Estado: Estado Local, Estado Global y Caché de Estado Remoto . . . 20

8. Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter **22**
8.1. Arquitectura de Frameworks Backend: Middleware, Inyección de Dependencias y Ciclo HTTP y Comparativa de Ecosistemas Backend: Productividad, Tipado y Rendimiento Extremo . . . 22
8.2. Aplicaciones de Escritorio: Qt (C++), .NET MAUI, Electron y Tauri (Rust + WebView) y Desarrollo Móvil: SwiftUI (iOS), Jetpack Compose (Android), Flutter y React Native . 22

A. Notación **25**

B. Hoja de fórmulas **26**

C. Soluciones **27**
C.1. HTML Living Standard y Accesibilidad Web (WCAG / ARIA) . . . 27
C.2. CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries . . . 27
C.3. Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf . . . 28
C.4. Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF) . . . 28
C.5. Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC . . . 29
C.6. Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis . . . 30
C.7. Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js . . . 30
C.8. Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter . . . 31

D. Auditoría de fuentes **32**

E. Plan de estudio **34**

Parte I

Partes VI–VIII: Web, Almacenamiento y Ecosistemas de Producción

Capítulo 1

HTML Living Standard y Accesibilidad Web (WCAG / ARIA)

Estudio riguroso de HTML como lenguaje de marcado declarativo (no de programación de propósito general), su gramática, ciclo de parseo en el navegador y validación accesible.

Consejo

La API de validación de restricciones de HTML5 verifica tipos, patrones regulares y rangos antes del envío HTTP, pero jamás sustituye la validación en el servidor.

Definición

Naturaleza declarativa de HTML y el estándar vivo WHATWG: HTML es un lenguaje formal de marcado estructural e hipertextual, no un lenguaje de programación Turing-completo.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Los atributos async, defer y type='module' desacoplan la descarga de scripts del bloqueo del parser HTML.

1.1 Naturaleza declarativa de HTML y el estándar vivo WHATWG y Semántica estructural, árbol de accesibilidad (AOM) y WAI-ARIA

A diferencia de un lenguaje de programación imperativo o funcional, HTML (HyperText Markup Language, mantenido como Living Standard por WHATWG) describe de manera declarativa la estructura semántica de un documento mediante elementos delimitados por etiquetas, atributos clave-valor y referencias de entidades. Carece de variables mutables, bucles condicionales o abstracción funcional general. El analizador sintáctico de HTML5 implementa una máquina de estados finitos con reglas deterministas de recuperación de errores: incluso ante etiquetas mal anidadas, construye el mismo árbol DOM en todos los motores conformes (Blink, WebKit, Gecko).

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Panel de Telemetría Industrial</title>
</head>
<body>
  <header role="banner">
    <h1>Estado del Clúster Principal</h1>
  </header>
  <main id="contenido-principal">
    <article aria-labelledby="sec-latencia">
      <h2 id="sec-latencia">Métricas de Latencia p99</h2>
      <p>Latencia actual: <data value="4.2">4,2 ms</data></p>
    </article>
  </main>
</body>
</html>
```

El navegador construye en paralelo al DOM el Árbol de Accesibilidad (Accessibility Object Model), consultado por lectores de pantalla mediante APIs nativas (IAccessible2, UIAutomation, NSAccessibility). La primera regla de WAI-ARIA establece que siempre debe preferirse un elemento HTML nativo (`<button>`, `<nav>`, `<main>`, `<dialog>`) sobre un `<div>` genérico con `role="button"`, ya que el elemento nativo proporciona automáticamente navegación por teclado (`Tab`, `Enter`, `Space`), gestión de foco y estados implícitos. Como se ilustra en la figura 1, el DOM alimenta tanto el Render Tree visual como la estructura semántica.

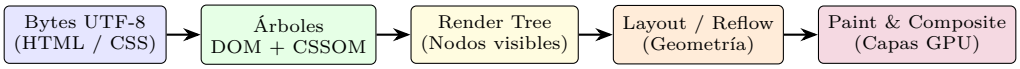


Figura 1. Tubería de renderizado del navegador (Critical Rendering Path): desde el flujo de bytes UTF-8 hasta el árbol DOM, CSSOM, Render Tree, Layout y Composite.

1.2 Formularios nativos, validación de restricciones y seguridad en enlaces y Carga eficiente de recursos, módulos ES y Web Components

Los controles de formulario (`<input>`, `<select>`, `<textarea>`) asociados explícitamente mediante `<label for="id">` exponen validación declarativa (`required`, `type="email"`, `min`, `max`, `pattern`). No obstante, dado que cualquier cliente puede falsificar peticiones HTTP ignorando el navegador, toda validación en el cliente es estrictamente una mejora de experiencia de usuario (UX) y el servidor debe revalidar íntegramente la carga útil. Asimismo, los enlaces externos con `target="_blank"` deben incluir `rel="noopener noreferrer"` para aislar el contexto de navegación y evitar ataques de tabnabbing inverso. Cuando el parser HTML encuentra una etiqueta `<script>` clásica sin atributos, detiene la construcción del DOM hasta descargar y ejecutar el script. El atributo `defer` descarga el script en paralelo y pospone su ejecución en orden hasta que el DOM está completo (`DOMContentLoaded`), mientras que `type="module"` activa el sistema de módulos ECMAScript en modo estricto con diferimiento implícito. Adicionalmente, la especificación HTML incluye `<template>` y `<slot>` como base de Web Components y Shadow DOM para encapsular estructura y estilo.

Problema resuelto

Enunciado Una página HTML descarga un documento base en 40 ms, una hoja CSS bloqueante de renderizado en 65 ms en paralelo con un script clásico bloqueante de 110 ms, y ejecuta el layout final en 15 ms tras ambos recursos. Calcule el tiempo total hasta el First Contentful Paint (FCP) en milisegundos.

Datos

- $t_{html} = 40\,\text{ms}$ — Tiempo de descarga del HTML base
- $t_{css} = 65\,\text{ms}$ — Tiempo de descarga de la hoja CSS
- $t_{js} = 110\,\text{ms}$ — Tiempo del script bloqueante mayor
- $t_{layout} = 15\,\text{ms}$ — Tiempo de Layout y Paint

Incógnita Tiempo total hasta First Contentful Paint T_fcp en ms

Modelo físico

Cálculo del tiempo de bloqueo del Critical Rendering Path

$$T_{\text{fcp}} = 40 + 110 + 15$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos el tiempo base HTML, el máximo de las descargas en paralelo (max(65, 110) = 110 ms) y el tiempo de layout

$$T_{\text{fcp}} = 40 + 110 + 15$$

$$((40 + 110) + 15)$$

Resultado

$$\textit{Resultado} = 165$$

Interpretación El First Contentful Paint ocurre a los 165 ms; añadir defer al script reduciría la espera crítica de recursos de 110 ms a 65 ms.
Comprobación de los pasos

- Paso 1: 165 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) [CH-01.1] Calcule (Reducción de latencia FCP al diferir un script de 110 ms dejando solo el CSS de 65 ms (40 + 65 + 15)) expresado en ms.
2 [PRB-0004] (Intermedio, Desarrollo por pasos) [CH-01.2] Calcule (Tamaño total en KiB de un documento HTML de 18 KiB más 3 iconos SVG inline de 4 KiB cada uno) expresado en KiB.
3 [PRB-0005] (Intermedio, Desarrollo por pasos) [CH-01.3] Calcule (Número total de nodos DOM generados por una tabla de 25 filas con 4 celdas por fila más el nodo tr por fila) expresado en nodos.
4 [PRB-0006] (Avanzado, Diseño) [CH-01.4] Calcule (Ahorro porcentual de transferencia al comprimir con Brotli un HTML de 80 KiB a 16 KiB) expresado en %.

Capítulo 2

CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries

Monografía técnica de CSS como lenguaje declarativo de presentación visual, geometría de cajas, contextos de apilamiento, diseño adaptable e ingeniería de sistemas de diseño.

Consejo

Flexbox distribuye espacio a lo largo de un eje principal dinámico, mientras que CSS Grid impone una malla bidimensional explícita de filas y columnas.

Definición

Algoritmo de la Cascada, Capas (@layer) y Especificidad Lexicográfica: El motor CSS resuelve conflictos entre declaraciones mediante una jerarquía estricta donde el origen y las capas (@layer) preceden a la especificidad del selector.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Animar exclusivamente transform y opacity evita recalcular el Layout (Reflow) y el Paint, delegando la interpolación al hilo compositor de la GPU.

2.1 Algoritmo de la Cascada, Capas (@layer) y Especificidad Lexicográfica y Modelo de Caja, Box-Sizing y Propiedades Lógicas Internacionalizadas

Para cada propiedad de cada elemento DOM, el motor CSS filtra las declaraciones aplicables y elige un único valor ganador evaluando en orden: 1) Origen e importancia (`!important` de usuario/autor), 2) Contexto de encapsulación (Shadow DOM), 3) Capas de cascada explícitas (`@layer reset, base, components, utilities`), 4) Especificidad del selector expresada como el vector lexicográfico $S = (a, b, c)$ donde a cuenta selectores de ID (`#nav`), b cuenta clases, atributos y pseudoclases (`.activo`, `[type="text"]`, `:hover`, mientras `:where()` aporta 0 y `:is()` toma el máximo de su lista), y c cuenta selectores de tipo y pseudoelementos (`article`, `::before`), y finalmente 5) Orden de aparición en el código fuente.

```
@layer base, components;

@layer base {
  :root {
    --espacio-base: 1rem;
    --color-superficie: oklch(98% 0.01 250);
  }
  *, *::before, *::after {
    box-sizing: border-box;
  }
}

@layer components {
  .panel-metricas {
    container-type: inline-size;
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(min(100%, 18rem), 1fr));
```

```
    gap: var(--espacio-base);
    background: var(--color-superficie);
  }
}
```

En el modelo tradicional `content-box`, el ancho visual ocupado por una caja es $W_{\text{total}} = W_{\text{content}} + 2P + 2B$. Al declarar `box-sizing: border-box`, el ancho declarado fija directamente W_{total}, descontando `padding` y `border` del área interior disponible. Además, el CSS moderno reemplaza propiedades físicas (`margin-left`, `width`) por propiedades lógicas relativas al flujo de escritura (`margin-inline-start`, `inline-size`, `block-size`), soportando automáticamente idiomas de derecha a izquierda (RTL) como árabe o hebreo sin duplicar hojas de estilo.

2.2 Contextos de Formato: Flexbox Unidimensional y CSS Grid Bidimensional y Container Queries, Rendimiento de Animaciones y Compositor GPU

Un contenedor `display: flex` calcula el tamaño base (`flex-basis`) de sus hijos directos y reparte el espacio libre positivo según los factores `flex-grow` o absorbe el déficit según `flex-shrink` ponderado por el tamaño base. Por su parte, `display: grid` permite definir pistas elásticas con la unidad fraccional `fr`, funciones `minmax()` y alineación de subrejillas mediante `subgrid`, permitiendo que componentes anidados alineen sus cabeceras y pies exactamente con la retícula del contenedor abuelo. Modificar propiedades geométricas (`width`, `top`, `margin`) en cada fotograma invalida el árbol de layout y obliga a la CPU a recalcular posiciones y repintar píxeles, provocando caídas por debajo de 60 FPS (presupuesto de 16,67 ms por fotograma). En cambio, `transform: translate3d(...)` y `opacity` operan sobre texturas ya rasterizadas en la GPU.

Problema resuelto

Enunciado Un contenedor Flex tiene un ancho interior de 960 px y contiene dos paneles con flex-basis de 200 px y 160 px respectivamente. Sus factores flex-grow son 2 y 1. Calcule el ancho final en píxeles del primer panel tras repartir el espacio libre.

Datos

- $W_{cont} = 960\,\text{px}$ — Ancho interior del contenedor Flex
- $B_1 = 200\,\text{px}$ — Tamaño base flex-basis del panel 1
- $B_2 = 160\,\text{px}$ — Tamaño base flex-basis del panel 2
- $g_1 = 2$ — Factor flex-grow del panel 1
- $g_2 = 1$ — Factor flex-grow del panel 2

Incógnita Ancho final del primer panel W_1 en px

Modelo físico

Distribución proporcional de espacio libre en Flexbox

$$W_1 = 200 + \frac{2}{2+1} \cdot (960 - 200 - 160)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el espacio libre (960 - 200 - 160 = 600 px) y asignamos dos tercios al primer panel

$$W_1 = 200 + \frac{2}{2+1} \cdot (960 - 200 - 160)$$

$$\left(200 + \frac{(2 \cdot ((960 - 200) - 160))}{3}\right)$$

Resultado

$$Resultado = 600$$

Interpretación El primer panel alcanza un ancho final exacto de 600 px y el segundo panel ocupa los 360 px restantes.
Comprobación de los pasos

- Paso 1: 600 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) [CH-02.1] Calcule (Ancho final del segundo panel en el contenedor Flex de 960 px (160 + 600 / 3)) expresado en px.
2 [PRB-0010] (Intermedio, Desarrollo por pasos) [CH-02.2] Calcule (Valor entero codificado en base 100 de la especificidad (1, 2, 3) para comparación rápida) expresado en puntos.
3 [PRB-0011] (Intermedio, Desarrollo por pasos) [CH-02.3] Calcule (Ancho de una columna 1fr en un CSS Grid de 1200 px con 3 columnas iguales y 2 huecos (gap) de 24 px) expresado en px.
4 [PRB-0012] (Avanzado, Diseño) [CH-02.4] Calcule (Ancho de área de contenido en border-box para una caja de 320 px con padding de 16 px y borde de 2 px por lado) expresado en px.

Capítulo 3

Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf

Estudio comparativo y riguroso de los lenguajes de serialización, configuración y marcado ligero empleados en sistemas distribuidos e infraestructura.

Consejo

Todo analizador XML en producción debe deshabilitar la resolución de entidades externas (XXE) y la expansión recursiva de DTD.

Definición

JSON (RFC 8259) y Validación Contractual con JSON Schema: JSON proporciona una gramática libre de contexto mínima y determinista, ideal para APIs web cuando se acompaña de validación estructural mediante JSON Schema.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Protocol Buffers reduce entre 3 y 10 veces el tamaño de carga útil y el tiempo de CPU frente a JSON al codificar campos con etiquetas numéricas y enteros Varint.

3.1 JSON (RFC 8259) y Validación Contractual con JSON Schema y YAML 1.2 vs TOML v1.0: Configuración Humana y Trampas Semánticas

Especificado en el RFC 8259, JSON admite únicamente seis tipos de valores: objetos (pares clave-valor con claves de tipo cadena), arrays ordenados, cadenas UTF-8, números en punto flotante IEEE 754, booleanos (`true`, `false`) y `null`. No admite comentarios, comas finales ni referencias cíclicas. Un riesgo clásico de interoperabilidad ocurre cuando enteros de 64 bits (`int64` mayores que $2^{53}-1$) se deserializan en motores JavaScript, perdiendo precisión en el mantisa de 53 bits; por ello, los identificadores de 64 bits deben serializarse como cadenas textuales.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["id_transaccion", "importe_centimos", "moneda"],
  "properties": {
    "id_transaccion": { "type": "string", "pattern": "^[0-9a-fA-F-]{36}$" },
    "importe_centimos": { "type": "integer", "minimum": 1 },
    "moneda": { "enum": ["EUR", "USD", "GBP"] }
  },
  "additionalProperties": false
}
```

YAML 1.2 se utiliza masivamente en Kubernetes, Ansible y pipelines CI/CD gracias a su sintaxis concisa, soporte de comentarios y reutilización mediante anclas (`&base`) y alias (`*base`). Sin embargo, presenta riesgos conocidos: en YAML 1.1 literales sin comillas como `NO` (código ISO de Noruega) u `on` se convertían silenciosamente en booleanos (`false`, `true`), y la carga insegura de etiquetas arbitrarias (`!!python/object/apply`) permite ejecución remota de código si no se invoca exclusivamen-

te `yaml.safe_load`. Por contraste, TOML (`Cargo.toml`, `pyproject.toml`) utiliza secciones explícitas `[tabla]` y tipos literales sin ambigüedad de indentación.

3.2 XML, Namespaces, XSD y Prevención de Ataques XXE y CSV (RFC 4180), Markdown (CommonMark) y Serialización Binaria con Protocol Buffers

XML sigue siendo el estándar normativo en facturación electrónica, banca (ISO 20022) y federación de identidad (SAML 2.0) gracias a sus espacios de nombres (`xmlns`) y validación formal con XML Schema (XSD). No obstante, su soporte histórico de Document Type Definitions (DTD) habilita vulnerabilidades críticas: la inyección de Entidades Externas XML (XXE) permite leer archivos locales del servidor (`file:///etc/passwd`), y la bomba de entidades recursivas (Billion Laughs) agota gigabytes de memoria RAM a partir de menos de 1 KiB de entrada. Mientras CSV (RFC 4180) requiere escapar comillas dobles y neutralizar fórmulas iniciales (`=`, `+`, `-`, `@`) para evitar CSV Injection en hojas de cálculo, y CommonMark estandariza la gramática de Markdown para documentación técnica, las comunicaciones de alta frecuencia entre microservicios emplean formatos binarios con esquema como Protocol Buffers (Protobuf v3). En Protobuf, cada campo se identifica en el flujo binario mediante un único byte de cabecera `(field_number « 3) | wire_type`, permitiendo evolucionar esquemas añadiendo nuevos campos sin romper clientes antiguos.

Problema resuelto

Enunciado Un mensaje JSON de telemetría ocupa 140 bytes en texto UTF-8. Al codificar los mismos 4 campos numéricos en Protocol Buffers v3, cada campo requiere 1 byte de etiqueta (tag) y 3 bytes de valor Varint promedio. Calcule el ahorro porcentual de ancho de banda logrado por Protobuf.
Datos

- $B_{json} = 140\,\text{bytes}$ — Tamaño del mensaje en JSON
- $n_{campos} = 4\,\text{campos}$ — Número de campos del mensaje
- $B_{campo} = 4\,\text{bytes}$ — Bytes por campo en Protobuf (1 tag + 3 valor)

Incógnita Ahorro porcentual de bytes A_pct en %
Modelo físico
Compresión binaria de un mensaje de telemetría con Protocol Buffers frente a JSON

$$A_{\text{pct}} = \frac{140 - 4 \cdot 4}{140} \cdot 100$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el tamaño en Protobuf (4 * 4 = 16 bytes) y el porcentaje de reducción respecto a 140 bytes

$$A_{\text{pct}} = \frac{140 - 4 \cdot 4}{140} \cdot 100$$

$$\frac{((140 - (4 \cdot 4)) \cdot 100)}{140}$$

Resultado

$$\textit{Resultado} = 88{,}5714285714$$

Interpretación Protocol Buffers reduce el tamaño del mensaje de 140 bytes a 16 bytes, ahorrando aproximadamente un 88,57 % del ancho de banda.
Comprobación de los pasos

- Paso 1: 88,5714 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0015**] **(Básico, Numérico)** [CH-03.1] Calcule (Tamaño en bytes del mensaje Protobuf con 4 campos de 4 bytes cada uno) expresado en bytes.
2 [**PRB-0016**] **(Intermedio, Desarrollo por pasos)** [CH-03.2] Calcule (Valor del byte de cabecera Protobuf para el campo número 3 con $wire_type2$(length-delimited: 3*8 + 2)) expresado en entero.
3 [**PRB-0017**] **(Intermedio, Desarrollo por pasos)** [CH-03.3] Calcule (Codificación ZigZag de sint32 para el entero negativo -15 (-2 * -15 - 1)) expresado en entero.
4 [**PRB-0018**] **(Avanzado, Diseño)** [CH-03.4] Calcule (Tráfico diario ahorrado en MB al enviar 10 millones de mensajes de 16 B en lugar de 116 B (dividiendo entre 10^6)) expresado en MB.

Capítulo 4

Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF)

Fundamentos matemáticos del modelo relacional de E. F. Codd, equivalencia entre álgebra y cálculo relacional, cierre de atributos y teoría formal de la normalización.

Consejo

Un esquema relacional en BCNF elimina toda redundancia derivada de dependencias funcionales al exigir que todo determinante no trivial sea superclave.

Definición

El Modelo Relacional de Codd y las Claves Relacionales: Una relación matemática es un subconjunto finito del producto cartesiano de dominios atómicos donde el orden de tuplas y atributos carece de significado semántico.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: La descomposición de R en R1 y R2 es una reunión sin pérdida si y solo si (R1 intersect R2) es superclave de R1 o de R2.

4.1 El Modelo Relacional de Codd y las Claves Relacionales y Dependencias Funcionales, Axiomas de Armstrong y Cierre de Atributos

Formulado por Edgar F. Codd en 1970, el modelo relacional independiza la representación lógica de los datos respecto a su disposición física en almacenamiento secundario. Dado un esquema $R(A_1, A_2, \ldots, A_n)$, un subconjunto de atributos $K \subseteq R$ es una superclave si no pueden existir dos tuplas distintas $t_1 \neq t_2$ en ninguna instancia válida $r(R)$ tales que $t_1[K] = t_2[K]$. Una superclave mínima (ningún subconjunto propio suyo es superclave) constituye una clave candidata, de las cuales se designa una clave primaria (PK) sujeta a integridad de entidad (ningún atributo primo puede ser `NULL`). Una dependencia funcional $X \to Y$ afirma que los valores del conjunto de atributos X determinan unívocamente los valores de Y. Los tres axiomas de Armstrong (reflexividad, aumentatividad y transitividad) son correctos y completos para inferir todas las dependencias implicadas F^+. En la práctica ingenieril, en lugar de enumerar F^+ (cuyo tamaño puede ser exponencial), calculamos el cierre de atributos X^+ partiendo de resultado $= X$ y añadiendo iterativamente W cada vez que $V \to W \in F$ cumple $V \subseteq$ resultado.

4.2 Anomalías de Diseño y Jerarquía de Formas Normales (1NF, 2NF, 3NF y BCNF) y Descomposición sin Pérdida (Lossless-Join) y Preservación de Dependencias

Mezclar múltiples entidades independientes en una sola tabla provoca anomalías de inserción (no poder registrar un departamento sin empleados), actualización (modificar un dato repetido en miles de filas con riesgo de inconsistencia) y borrado. La Primera Forma Normal (1NF) exige dominios atómicos; la 2NF prohíbe dependencias parciales de atributos no primos respecto a una clave compuesta; la 3NF prohíbe dependencias transitivas entre atributos no primos; y la Forma Normal de Boyce-Codd (BCNF) exige que para toda $X \to Y$ no trivial, X sea superclave. Dividir una tabla sin verificar el teorema de Heath

produce descomposiciones con pérdida (lossy joins) donde la reunión natural $R_1 \bowtie R_2$ genera tuplas espurias inexistentes en la relación original. Mientras que toda relación admite una descomposición en BCNF sin pérdida de información, en ocasiones aislar todas las dependencias en tablas individuales sólo es posible en 3NF mediante el algoritmo de síntesis de recubrimiento canónico.

Problema resuelto

Enunciado Una tabla desnormalizada de líneas de pedido con 50 000 filas repite en cada fila los 120 bytes de datos del cliente (habiendo solo 2 000 clientes distintos). Al descomponer a BCNF en Lineas(50 000 filas) y Clientes(2 000 filas x 120 bytes), calcule el espacio total en KiB (1 KiB = 1024 B) ahorrado por no repetir los 120 bytes en las 48 000 filas excedentes.
Datos

- $N_{exc} = 48000\,\text{filas}$ — Filas donde se repetían los datos del cliente
- $B_{cli} = 120\,\text{bytes}$ — Bytes redundantes por fila
- $K = 1024\,\text{bytes}\,\text{KiB}^{-1}$ — Bytes por kibibyte

Incógnita Espacio ahorrado S_kib en KiB
Modelo físico
Eliminación de redundancia mediante descomposición a BCNF

$$S_{\text{kib}} = \frac{48000 \cdot 120}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 48 000 filas redundantes por 120 bytes y dividimos entre 1024

$$S_{\text{kib}} = \frac{48000 \cdot 120}{1024}$$

$$\frac{(48000 \cdot 120)}{1024}$$

Resultado

$$\mathit{Resultado} = 5625$$

Interpretación La normalización a BCNF ahorra exactamente 5 625 KiB de almacenamiento redundante y elimina las anomalías de actualización.
Comprobación de los pasos

- Paso 1: 5625 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) [CH-04.1] Calcule (Espacio en KiB ocupado por la nueva tabla Clientes con 2048 filas de 128 bytes (2048 * 128 / 1024)) expresado en KiB.
2 [PRB-0022] (Intermedio, Desarrollo por pasos) [CH-04.2] Calcule (Número de superclaves en una relación de 5 atributos cuya única clave candidata tiene 2 atributos (2^(5-2))) expresado en superclaves.
3 [PRB-0023] (Intermedio, Desarrollo por pasos) [CH-04.3] Calcule (Tuplas espurias generadas al hacer join de dos proyecciones mal descompuestas que producen 140 filas frente a 100 reales)

expresado en tuplas.
4 [PRB-0024] (Avanzado, Diseño) [CH-04.4] Calcule (Número de atributos en la intersección R1 n R2 si |R1|=4, |R2|=5 y |R1 U R2|=7) expresado en atributos.

Capítulo 5

Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC

Ingeniería interna de sistemas gestores de bases de datos (SGBD): Buffer Pool, Write-Ahead Logging (ARIES), optimizadores basados en costes y control de concurrencia multiversión.

Consejo

La regla WAL garantiza atomicidad y durabilidad exigiendo que todo registro de log (UNDO/REDO) llegue a almacenamiento no volátil antes de confirmar la transacción o volcar la página sucia.

Definición

Árboles B+ (B+-Trees) frente a LSM-Trees (Log-Structured Merge-Trees): Un árbol B+ con factor de ramificación B=250 localiza cualquier clave entre 1 000 millones de filas en tan solo 4 lecturas de página.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: El aislamiento por instantáneas (Snapshot Isolation / Repeatable Read en PostgreSQL) evita lecturas sucias, no repetibles y fantasmas, pero permite Write Skew si no se emplea Serializable Snapshot Isolation (SSI).

5.1 Árboles B+ (B+-Trees) frente a LSM-Trees (Log-Structured Merge-Trees) y Diseño de Índices Compuestos, Índices Cubrientes (Covering) y Optimizadores de Costes

Los motores relacionales clásicos (PostgreSQL, InnoDB, SQLite) organizan tablas e índices en páginas de disco de 8 KiB o 16 KiB estructuradas como árboles B+, como muestra la figura 2. Todos los valores residen en el nivel de las hojas, las cuales están doblemente enlazadas para acelerar escaneos por rango (`BETWEEN`). En contrapartida, motores de alta tasa de ingesta (RocksDB, Cassandra, ScyllaDB) emplean LSM-Trees: toda escritura se registra secuencialmente en un log en disco y en una estructura ordenada en RAM (`MemTable`); al llenarse, se vuelca como un archivo inmutable (`SSTable`) compactado en segundo plano y acelerado mediante filtros de Bloom. El optimizador de consultas (Query Planner) utiliza histogramas de distribución, cardinalidad (NDV) y factores de correlación física para estimar el coste en E/S y CPU de distintos planes de ejecución (`Index Scan`, `Bitmap Heap Scan`, `Seq Scan`, `Hash Join`, `Merge Join`, `Nested Loop`). Cuando un índice incluye todas las columnas requeridas por una consulta (por ejemplo, `CREATE INDEX idx_pedidos ON pedidos (cliente_id, fecha) INCLUDE (total)`), el motor ejecuta un `Index-Only Scan` sin necesidad de acceder a las páginas de la tabla principal (Heap).

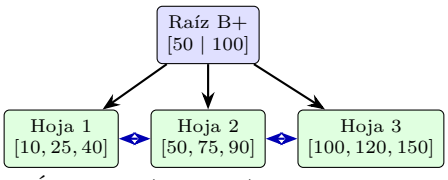


Figura 2. Estructura de un índice Árbol B+ (B+-Tree): los nodos internos dirigen la búsqueda por claves separadoras y las hojas enlazadas almacenan los punteros o tuplas en orden secuencial.

5.2 Transacciones ACID, Write-Ahead Logging (WAL) y Protocolo ARIES y Niveles de Aislamiento, Anomalías de Concurrencia y MVCC (Snapshot Isolation)

Las garantías ACID comprenden Atomicidad (todo o nada mediante registros UNDO), Consistencia (preservación de invariantes e integridad referencial), Aislamiento (ejecución concurrente equivalente a una planificación controlada) y Durabilidad (persistencia mediante registros REDO en el Write-Ahead Log). Tras un fallo eléctrico, el algoritmo de recuperación ARIES ejecuta tres fases a partir del último punto de control (Checkpoint): Análisis, REDO (repitiendo la historia hasta el estado exacto previo al fallo) y UNDO (revirtiendo transacciones inconclusas). Bajo MVCC, cada transacción recibe un identificador monotónico (`txid`) y cada actualización inserta una nueva versión de la tupla marcada con el intervalo de validez `[xmin, xmax)`. Así, las lecturas leen una instantánea consistente sin bloquear a los escritores. Sin embargo, dos transacciones concurrentes que leen un mismo invariante y escriben en filas disjuntas pueden cometer la anomalía de asimetría de escritura (`Write Skew`); prevenirla requiere bloqueos explícitos (`SELECT ... FOR UPDATE`) o el algoritmo SSI (`SERIALIZABLE`).

Problema resuelto

Enunciado En una base de datos con páginas de 8 KiB, cada nodo interno de un índice B+ tiene un factor de ramificación efectivo de 200 hijos y cada página hoja almacena 80 entradas de índice. Calcule el número total de filas (en millones) que puede indexar un árbol B+ de altura h = 3 (1 raíz, 1 nivel intermedio de nodos internos y 1 nivel de hojas).

Datos

- $B_{int} = 200\,\text{hijos}\,\text{nodo}^{-1}$ — Factor de ramificación de nodos internos
- $C_{hoja} = 80\,\text{filas}\,\text{hoja}^{-1}$ — Entradas almacenadas por página hoja

Incógnita Capacidad total en millones de filas $N_millones$

Modelo físico

Capacidad de direccionamiento de un Árbol B+ de 3 niveles

$$N_{\text{millones}} = \frac{200 \cdot 200 \cdot 80}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 200 hijos de la raíz por los 200 hijos del nivel intermedio por las 80 filas por hoja y dividimos entre 10^6

$$N_{\text{millones}} = \frac{200 \cdot 200 \cdot 80}{10^6}$$

$$\frac{((200 \cdot 200) \cdot 80)}{1000000}$$

Resultado

$$Resultado = 3{,}2$$

Interpretación Con solo 3 niveles (y los 2 primeros en RAM), el árbol B+ direcciona hasta 3,2 millones de filas requiriendo como máximo una única lectura física de disco para la hoja.

Comprobación de los pasos

- Paso 1: 3,2 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0027**] **(Básico, Numérico)** [CH-05.1] Calcule (Número de páginas hoja en el árbol B+ de 3 niveles con factor de ramificación 200 (200 * 200)) expresado en hojas.
2 [**PRB-0028**] **(Intermedio, Desarrollo por pasos)** [CH-05.2] Calcule (Memoria RAM en KiB requerida para mantener en Buffer Pool la raíz y los 200 nodos internos de 8 KiB ((1 + 200) * 8)) expresado en KiB.
3 [**PRB-0029**] **(Intermedio, Desarrollo por pasos)** [CH-05.3] Calcule (Coste estimado en unidades de página de un Index Scan que lee 3 niveles de índice más 12 páginas de Heap) expresado en páginas.
4 [**PRB-0030**] **(Avanzado, Diseño)** [CH-05.4] Calcule (Tasa de escritura WAL en MB/s para 5 000 transacciones por segundo que generan 400 bytes de log cada una) expresado en MB/s.

Capítulo 6

Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis

Arquitectura de bases de datos a escala: replicación síncrona/asíncrona, particionamiento por hash consistente y rango, transacciones distribuidas y motores de producción.

Consejo

PostgreSQL combina relacionalidad estricta con tipos JSONB indexados (GIN), mientras que SQLite en modo WAL ofrece lecturas concurrentes embebidas de latencia sub-milisegundo.

Definición

Teorema CAP y Clasificación PACELC en Almacenamiento Distribuido: El modelo PACELC extiende el teorema CAP demostrando que incluso en ausencia de particiones de red existe una disyuntiva fundamental entre latencia (L) y consistencia (C).

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Los almacenes documentales eliminan el coste de joins cuando el agregado se consulta íntegro, y Redis proporciona estructuras atómicas en RAM con latencia de microsegundos.

6.1 Teorema CAP y Clasificación PACELC en Almacenamiento Distribuido y Replicación Líder-Seguidor, Quórums de Lectura/Escritura y Sharding

El teorema CAP (Brewer, demostrado formalmente por Gilbert y Lynch) establece que ante una partición de red (P) entre nodos, un almacén de datos distribuido debe elegir entre cancelar la operación para preservar la Consistencia linealizable (CP) o responder con datos potencialmente desactualizados para preservar la Disponibilidad (AP). El teorema PACELC (Abadi) añade que, incluso cuando el sistema opera normalmente (Else), debe decidirse entre replicar síncronamente (penalizando la Latencia L a cambio de Consistencia C) o asíncronamente. Cuando el volumen de escrituras o el conjunto de datos supera la capacidad de un único nodo, los datos se dividen horizontalmente en particiones (Shards). El particionamiento por rango facilita consultas secuenciales pero puede sufrir puntos calientes (hotspots) si todas las escrituras llevan la marca temporal actual; el particionamiento por Hash Consistente (con nodos virtuales en un anillo de 2^{32} posiciones) distribuye uniformemente la carga y minimiza el movimiento de claves al añadir o retirar servidores.

6.2 PostgreSQL y SQLite en Producción: De Sistemas Embebidos a Escala Empresarial y Motores NoSQL Especializados: Documentos (MongoDB) y Memoria Clave-Valor (Redis)

PostgreSQL constituye el estándar de referencia en ingeniería de datos moderna: soporta índices B-Tree, Hash, GiST, SP-GiST, GIN (para documentos `JSONB` y búsqueda de texto completo) y BRIN (para series temporales masivas), además de particionamiento declarativo y replicación lógica. Por su parte, SQLite ejecuta el motor transaccional completo dentro del espacio de direcciones del proceso anfitrión sin latencia

de red; al activar `PRAGMA journal_mode = WAL;` y `PRAGMA synchronous = NORMAL;`, alcanza decenas de miles de operaciones por segundo en un único archivo de disco. MongoDB almacena documentos en formato binario BSON sobre el motor WiredTiger, resultando idóneo para catálogos polimórficos donde la entidad raíz y sus subdocumentos forman un agregado de consulta cohesivo. Redis (y alternativas compatibles como Valkey) mantiene estructuras de datos ricas en memoria (Strings, Hashes, Sorted Sets, Streams, HyperLogLog) sobre un bucle de eventos monohilo libre de contención de cerrojos, ejecutando scripts Lua de forma atómica para cachés distribuidas, colas de trabajo y limitadores de tasa (rate limiters).

Problema resuelto

Enunciado Un clúster de base de datos distribuida mantiene N = 5 réplicas de cada partición. Si las escrituras se configuran con un quórum de confirmación W = 3 nodos para tolerar la caída de 2 réplicas, calcule el número mínimo de nodos R que debe consultar una lectura para garantizar intersección estricta (W + R > N => R = N - W + 1).
Datos

- $N = 5\,\text{nodos}$ — Factor total de replicación
- $W = 3\,\text{nodos}$ — Quórum de escritura confirmada

Incógnita Quórum mínimo de lectura R en nodos
Modelo físico
Cálculo del quórum mínimo de lectura para consistencia fuerte

$$R = 5 - 3 + 1$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Aplicamos la condición de intersección de quórum estricto R = N - W + 1

$$R = 5 - 3 + 1$$

$$((5 - 3) + 1)$$

Resultado

$$\mathit{Resultado} = 3$$

Interpretación Basta con leer de R = 3 réplicas cualesquiera para garantizar que al menos una contiene la última escritura confirmada por W = 3 nodos.
Comprobación de los pasos

- Paso 1: 3 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) [CH-06.1] Calcule (Quórum de lectura requerido si N = 7 réplicas y W = 4 escrituras (7 - 4 + 1)) expresado en nodos.
2 [PRB-0034] (Intermedio, Desarrollo por pasos) [CH-06.2] Calcule (Porcentaje de claves redistribuidas por Hash Consistente al pasar de 9 a 10 nodos (100 / 10)) expresado en %.
3 [PRB-0035] (Intermedio, Desarrollo por pasos) [CH-06.3] Calcule (Memoria RAM total en GB para almacenar en Redis 50 millones de claves de 200 bytes promedio (dividiendo entre 10^9))

expresado en GB.
4 [PRB-0036] (Avanzado, Diseño) [CH-06.4] Calcule (Número máximo de fallos de nodo tolerados por un clúster Raft de 5 nodos ((5 - 1) / 2)) expresado en nodos.

Capítulo 7

Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js

Arquitectura interna y patrones de ingeniería de los principales frameworks de interfaz de usuario web.

Consejo

Vue 3 rastrea dependencias en tiempo de ejecución mediante objetos Proxy de ES6, mientras que Svelte compila las asignaciones reactivas en actualizaciones quirúrgicas del DOM en tiempo de build.

Definición

Del DOM Imperativo a la UI Declarativa: Función Estado -> Vista: Los frameworks frontend modernos modelan la interfaz como una proyección determinista V = f(S) del estado actual, automatizando la sincronización incremental del DOM.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Separar la caché de estado del servidor (TanStack Query / SWR) del estado efímero de la interfaz reduce drásticamente la complejidad de almacenes globales como Redux.

7.1 Del DOM Imperativo a la UI Declarativa: Función Estado -> Vista y React y Next.js: Reconciliación Fiber, Hooks, Server Components e Hidratación

Manipular manualmente nodos DOM ante eventos asíncronos conduce a transiciones de estado no controladas y fugas de memoria. Los frameworks modernos adoptan un paradigma declarativo donde el desarrollador describe cómo debe lucir la interfaz para cada estado S, y el motor de ejecución determina el conjunto mínimo de mutaciones DOM necesarias. Existen dos grandes estrategias: reconciliación de Árbol Virtual (Virtual DOM en React y Vue) y reactividad de grano fino basada en grafos de dependencias o compilación anticipada (Signals en Angular/Solid y compilador de Svelte). En React, los componentes funcionales son funciones puras que utilizan Hooks (`useState`, `useReducer`, `useMemo`, `useEffect`) almacenados en una lista enlazada asociada al nodo Fiber. En aplicaciones full-stack con Next.js (App Router), los React Server Components (RSC) se ejecutan exclusivamente en el servidor −accediendo directamente a bases de datos sin enviar su código JavaScript al navegador−, reservando los Client Components (`"use client"`) para islas de interactividad que requieren hidratación.

7.2 Angular, Vue y Svelte: Inyección de Dependencias, Proxies Reactivos y Compilación y Gestión de Estado: Estado Local, Estado Global y Caché de Estado Remoto

Angular ofrece una plataforma empresarial opinada en TypeScript con inyección de dependencias jerárquica, enrutamiento tipado, RxJS y reactividad moderna mediante `signal()` y `computed()`. Vue 3 combina la Composition API (`ref`, `reactive`) implementada sobre `Proxy` con un compilador de plantillas que marca banderas de parcheo (`PatchFlags`) para saltar subárboles estáticos. Svelte prescinde del Virtual DOM compilando cada componente en código JavaScript imperativo mínimo. Uno de los antipatrones más frecuentes en ingeniería frontend consiste en volcar respuestas HTTP crudas en un almacén global único sin políticas de invalidación, reintento ni deduplicación. La arquitectura moderna distingue estrictamente entre estado del servidor (gestionado mediante cachés declarativas con `stale-while-revalidate`

y actualización optimista), estado de la URL (filtros y paginación sincronizados con el router) y estado puramente local de la interfaz.

Problema resuelto

Enunciado Una aplicación SPA tradicional descarga un bundle monolítico de 480 KiB de JavaScript. Al migrar a una arquitectura híbrida SSR + Server Components, el 65 % del código de presentación y formateo permanece en el servidor. Calcule el tamaño en KiB del bundle JavaScript que aún debe descargar e hidratar el navegador.
Datos

- $B_{spa} = 480\,\text{KiB}$ — Tamaño original del bundle SPA
- $p_{srv} = 65\ \%$ — Porcentaje migrado a Server Components

Incógnita Tamaño del bundle cliente restante B_cli en KiB
Modelo físico
Reducción del bundle JavaScript mediante React Server Components y Code Splitting

$$B_{\text{cli}} = 480 \cdot \frac{100 - 65}{100}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Calculamos el 35 % restante del bundle original de 480 KiB

$$B_{\text{cli}} = 480 \cdot \frac{100 - 65}{100}$$

$$\frac{(480 \cdot (100 - 65))}{100}$$

Resultado

$$\mathit{Resultado} = 168$$

Interpretación El cliente solo descarga e hidrata 168 KiB de JavaScript, reduciendo el Time to Interactive (TTI) en dispositivos móviles.
Comprobación de los pasos

- Paso 1: 168 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0039] (Básico, Numérico) [CH-07.1] Calcule (Ahorro absoluto en KiB al reducir el bundle de 480 KiB a 168 KiB) expresado en KiB.
2 [PRB-0040] (Intermedio, Desarrollo por pasos) [CH-07.2] Calcule (Tiempo de parseo y compilación JS en móvil a razón de 1,5 ms por cada KiB para 168 KiB) expresado en ms.
3 [PRB-0041] (Intermedio, Desarrollo por pasos) [CH-07.3] Calcule (Nodos re-evaluados con Signals en una lista de 500 filas cuando cambia 1 celda frente a 500 sin memoización (ahorro)) expresado en nodos.
4 [PRB-0042] (Avanzado, Diseño) [CH-07.4] Calcule (Número de peticiones de red tras deduplicar 6 componentes que solicitan simultáneamente el mismo endpoint) expresado en peticiones.

Capítulo 8

Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter

Análisis arquitectónico de los ecosistemas de servidor de alto rendimiento y de las plataformas de interfaz gráfica de escritorio y dispositivos móviles.

Consejo

Tauri reduce en un orden de magnitud el consumo de memoria RAM y el tamaño del instalador frente a Electron al reutilizar el WebView nativo del sistema operativo y ejecutar el backend en Rust.

Definición

Arquitectura de Frameworks Backend: Middleware, Inyección de Dependencias y Ciclo HTTP: Todos los frameworks backend modernos estructuran el procesamiento de peticiones como una composición funcional de middlewares alrededor de un controlador tipado.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Flutter dibuja cada píxel en su propio lienzo GPU mediante el motor Impeller sin depender de widgets OEM, mientras que SwiftUI y Jetpack Compose ofrecen integración nativa declarativa.

8.1 Arquitectura de Frameworks Backend: Middleware, Inyección de Dependencias y Ciclo HTTP y Comparativa de Ecosistemas Backend: Productividad, Tipado y Rendimiento Extremo

Tanto en ecosistemas asíncronos orientados a eventos (Fastify/Express en Node.js, FastAPI sobre ASGI/Uvicorn en Python, Axum/Actix sobre Tokio en Rust, Phoenix sobre BEAM en Elixir) como en plataformas empresariales multihilo o con hilos virtuales (Spring Boot 3 en Java 21, ASP.NET Core en .NET 8, Django, Ruby on Rails, Laravel), cada petición HTTP atraviesa una cadena concéntrica de middlewares (trazabilidad, CORS, autenticación JWT/OIDC, rate limiting) antes de deserializar y validar el payload hacia un DTO tipado. En FastAPI (Python), las anotaciones de tipos estándar junto con Pydantic v2 (escrito en Rust) validan cargas JSON y generan automáticamente la especificación OpenAPI 3.1. En Spring Boot y ASP.NET Core, el contenedor de Inyección de Dependencias (IoC) gestiona el ciclo de vida de servicios (`Singleton`, `Scoped` por petición, `Transient`), facilitando las pruebas unitarias mediante inversión de dependencias. En Rust, Axum aprovecha el sistema de tipos para extraer parámetros y estado compartido (`State<Arc<AppState>>`) con coste de abstracción nulo.

8.2 Aplicaciones de Escritorio: Qt (C++), .NET MAUI, Electron y Tauri (Rust + WebView) y Desarrollo Móvil: SwiftUI (iOS), Jetpack Compose (Android), Flutter y React Native

Para aplicaciones de escritorio de misión crítica (CAD, instrumentación médica, edición de vídeo), Qt 6 en C++ proporciona renderizado acelerado por hardware y un sistema de señales y ranuras (`signals & slots`) nativo. Cuando se desea reutilizar tecnologías web en escritorio, Electron empaqueta una instancia

completa de Chromium y Node.js por aplicación (~150 MB en disco y cientos de MB de RAM base), mientras que Tauri delega la interfaz al WebView del sistema (WebKitGTK, WebView2, WKWebView) y comunica el frontend con un binario nativo en Rust mediante IPC tipado. El desarrollo móvil moderno ha convergido hacia interfaces declarativas reactivas: SwiftUI en el ecosistema Apple y Jetpack Compose en Android eliminan los antiguos archivos XML/Storyboards imperativos. En el ámbito multiplataforma, React Native sincroniza componentes JavaScript/TypeScript con vistas nativas OEM mediante la nueva arquitectura JSI (JavaScript Interface) sin puente serializado, mientras que Flutter (Dart AOT) controla directamente el pipeline gráfico con Impeller.

Problema resuelto

Enunciado Una estación de trabajo ejecuta simultáneamente 4 utilidades corporativas de escritorio. Empaquetadas con Electron, cada instancia consume 220 MB de memoria RAM debido a procesos Chromium independientes; migradas a Tauri (Rust + WebView compartido), cada instancia consume 45 MB. Calcule la memoria RAM total ahorrada en megabytes.
Datos

- $N_{apps} = 4\,\text{aplicaciones}$ — Número de aplicaciones ejecutadas
- $M_{elec} = 220\,\text{MB}$ — Memoria por aplicación en Electron
- $M_{tauri} = 45\,\text{MB}$ — Memoria por aplicación en Tauri

Incógnita Memoria RAM total ahorrada M_ahorro en MB
Modelo físico
Ahorro de memoria RAM en escritorio con Tauri frente a Electron

$$M_{\text{ahorro}} = 4 \cdot (220 - 45)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 4 aplicaciones por la diferencia de consumo unitario (220 - 45 = 175 MB)

$$M_{\text{ahorro}} = 4 \cdot (220 - 45)$$

$$(4 \cdot (220 - 45))$$

Resultado

$$Resultado = 700$$

Interpretación La migración de las 4 utilidades a Tauri libera exactamente 700 MB de memoria RAM en la estación de trabajo.
Comprobación de los pasos

- Paso 1: 700 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0045] (Básico, Numérico) [CH-08.1] Calcule (Reducción del tamaño de descarga en MB al pasar de un instalador Electron de 115 MB a uno Tauri de 8 MB) expresado en MB.
2 [PRB-0046] (Intermedio, Desarrollo por pasos) [CH-08.2] Calcule (Presupuesto de tiempo por fotograma en milisegundos (redondeado a centésimas x100) para una app Flutter a 120 Hz (100000

/ 125)) expresado en centésimas de ms.
3 [PRB-0047] (Intermedio, Desarrollo por pasos) [CH-08.3] Calcule (Capacidad total de peticiones por segundo en un clúster de 6 pods Axum que procesan 35 000 req/s cada uno) expresado en req/s.
4 [PRB-0048] (Avanzado, Diseño) [CH-08.4] Calcule (Instancias de un servicio Scoped creadas durante un lote de 250 peticiones HTTP si se inyecta 2 veces por petición) expresado en instancias.

Apéndice A

Notación

Símbolo	Significado	Unidades
$S = (a, b, c)$	Vector lexicográfico de especificidad CSS	adimensional
$X \to Y$	Dependencia funcional relacional en esquema R	relaciónlógica
$h = \lceil \log_B(N) \rceil$	Altura de un árbol B+ con factor de ramificación B	nivelesdepágina

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 HTML Living Standard y Accesibilidad Web (WCAG / ARIA)

Problema resuelto [PRB-0001]
Una página HTML descarga un documento base en 40 ms, una hoja CSS bloqueante de renderizado en 65 ms en paralelo con un script clásico bloqueante de 110 ms, y ejecuta el layout final en 15 ms tras ambos recursos. Calcule el tiempo total hasta el First Contentful Paint (FCP) en milisegundos.
Respuesta

$$Resultado = 165$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[CH-01.1] Calcule (Reducción de latencia FCP al diferir un script de 110 ms dejando solo el CSS de 65 ms (40 + 65 + 15)) expresado en ms.
Respuesta 120
Ejercicio 2 [PRB-0004]
[CH-01.2] Calcule (Tamaño total en KiB de un documento HTML de 18 KiB más 3 iconos SVG inline de 4 KiB cada uno) expresado en KiB.
Respuesta 30
Ejercicio 3 [PRB-0005]
[CH-01.3] Calcule (Número total de nodos DOM generados por una tabla de 25 filas con 4 celdas por fila más el nodo tr por fila) expresado en nodos.
Respuesta 125
Ejercicio 4 [PRB-0006]
[CH-01.4] Calcule (Ahorro porcentual de transferencia al comprimir con Brotli un HTML de 80 KiB a 16 KiB) expresado en %.
Respuesta 80

Cálculos reproducidos

$((40 + 110) + 15) = 165$

C.2 CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries

Problema resuelto [PRB-0007]
Un contenedor Flex tiene un ancho interior de 960 px y contiene dos paneles con flex-basis de 200 px y 160 px respectivamente. Sus factores flex-grow son 2 y 1. Calcule el ancho final en píxeles del primer panel tras repartir el espacio libre.
Respuesta

$$Resultado = 600$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]
[CH-02.1] Calcule (Ancho final del segundo panel en el contenedor Flex de 960 px (160 + 600 / 3)) expresado en px.
Respuesta 360

Ejercicio 2 [PRB-0010]
[CH-02.2] Calcule (Valor entero codificado en base 100 de la especificidad (1, 2, 3) para comparación rápida) expresado en puntos.
Respuesta 10203
Ejercicio 3 [PRB-0011]
[CH-02.3] Calcule (Ancho de una columna 1fr en un CSS Grid de 1200 px con 3 columnas iguales y 2 huecos (gap) de 24 px) expresado en px.
Respuesta 384
Ejercicio 4 [PRB-0012]
[CH-02.4] Calcule (Ancho de área de contenido en border-box para una caja de 320 px con padding de 16 px y borde de 2 px por lado) expresado en px.
Respuesta 284

Cálculos reproducidos

$\left(200 + \frac{(2\cdot((960-200)-160))}{3}\right) = 600$

C.3 Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf

Problema resuelto [PRB-0013]
Un mensaje JSON de telemetría ocupa 140 bytes en texto UTF-8. Al codificar los mismos 4 campos numéricos en Protocol Buffers v3, cada campo requiere 1 byte de etiqueta (tag) y 3 bytes de valor Varint promedio. Calcule el ahorro porcentual de ancho de banda logrado por Protobuf.
Respuesta

$$Resultado = 88{,}5714285714$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0015]
[CH-03.1] Calcule (Tamaño en bytes del mensaje Protobuf con 4 campos de 4 bytes cada uno) expresado en bytes.
Respuesta 16
Ejercicio 2 [PRB-0016]
[CH-03.2] Calcule (Valor del byte de cabecera Protobuf para el campo número 3 con $\text{wire}_t ype2$(length-delimited: 3*8 + 2)) expresado en entero.
Respuesta 26
Ejercicio 3 [PRB-0017]
[CH-03.3] Calcule (Codificación ZigZag de sint32 para el entero negativo -15 (-2 * -15 - 1)) expresado en entero.
Respuesta 29
Ejercicio 4 [PRB-0018]
[CH-03.4] Calcule (Tráfico diario ahorrado en MB al enviar 10 millones de mensajes de 16 B en lugar de 116 B (dividiendo entre 10^6)) expresado en MB.
Respuesta 1000

Cálculos reproducidos

$\frac{((140-(4\cdot4))\cdot100)}{140} = 88{,}5714285714$

C.4 Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF)

Problema resuelto [PRB-0019]

Una tabla desnormalizada de líneas de pedido con 50 000 filas repite en cada fila los 120 bytes de datos del cliente (habiendo solo 2 000 clientes distintos). Al descomponer a BCNF en Lineas(50 000 filas) y Clientes(2 000 filas x 120 bytes), calcule el espacio total en KiB (1 KiB = 1024 B) ahorrado por no repetir los 120 bytes en las 48 000 filas excedentes.
Respuesta

$$Resultado = 5625$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
[CH-04.1] Calcule (Espacio en KiB ocupado por la nueva tabla Clientes con 2048 filas de 128 bytes (2048 * 128 / 1024)) expresado en KiB.
Respuesta 256
Ejercicio 2 [PRB-0022]
[CH-04.2] Calcule (Número de superclaves en una relación de 5 atributos cuya única clave candidata tiene 2 atributos (2^(5-2))) expresado en superclaves.
Respuesta 8
Ejercicio 3 [PRB-0023]
[CH-04.3] Calcule (Tuplas espurias generadas al hacer join de dos proyecciones mal descompuestas que producen 140 filas frente a 100 reales) expresado en tuplas.
Respuesta 40
Ejercicio 4 [PRB-0024]
[CH-04.4] Calcule (Número de atributos en la intersección R1 n R2 si |R1|=4, |R2|=5 y |R1 U R2|=7) expresado en atributos.
Respuesta 2

Cálculos reproducidos

$\frac{(48000 \cdot 120)}{1024} = 5625$

C.5 Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC

Problema resuelto [PRB-0025]
En una base de datos con páginas de 8 KiB, cada nodo interno de un índice B+ tiene un factor de ramificación efectivo de 200 hijos y cada página hoja almacena 80 entradas de índice. Calcule el número total de filas (en millones) que puede indexar un árbol B+ de altura h = 3 (1 raíz, 1 nivel intermedio de nodos internos y 1 nivel de hojas).
Respuesta

$$Resultado = 3{,}2$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
[CH-05.1] Calcule (Número de páginas hoja en el árbol B+ de 3 niveles con factor de ramificación 200 (200 * 200)) expresado en hojas.
Respuesta 40000
Ejercicio 2 [PRB-0028]
[CH-05.2] Calcule (Memoria RAM en KiB requerida para mantener en Buffer Pool la raíz y los 200 nodos internos de 8 KiB ((1 + 200) * 8)) expresado en KiB.
Respuesta 1608
Ejercicio 3 [PRB-0029]
[CH-05.3] Calcule (Coste estimado en unidades de página de un Index Scan que lee 3 niveles de índice más 12 páginas de Heap) expresado en páginas.
Respuesta 15
Ejercicio 4 [PRB-0030]

[CH-05.4] Calcule (Tasa de escritura WAL en MB/s para 5 000 transacciones por segundo que generan 400 bytes de log cada una) expresado en MB/s.
Respuesta 2

Cálculos reproducidos

$\frac{((200 \cdot 200) \cdot 80)}{1000000} = 3{,}2$

C.6 Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis

Problema resuelto [PRB-0031]
Un clúster de base de datos distribuida mantiene N = 5 réplicas de cada partición. Si las escrituras se configuran con un quórum de confirmación W = 3 nodos para tolerar la caída de 2 réplicas, calcule el número mínimo de nodos R que debe consultar una lectura para garantizar intersección estricta (W + R > N => R = N - W + 1).
Respuesta

$$Resultado = 3$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
[CH-06.1] Calcule (Quórum de lectura requerido si N = 7 réplicas y W = 4 escrituras (7 - 4 + 1)) expresado en nodos.
Respuesta 4
Ejercicio 2 [PRB-0034]
[CH-06.2] Calcule (Porcentaje de claves redistribuidas por Hash Consistente al pasar de 9 a 10 nodos (100 / 10)) expresado en %.
Respuesta 10
Ejercicio 3 [PRB-0035]
[CH-06.3] Calcule (Memoria RAM total en GB para almacenar en Redis 50 millones de claves de 200 bytes promedio (dividiendo entre 10^9)) expresado en GB.
Respuesta 10
Ejercicio 4 [PRB-0036]
[CH-06.4] Calcule (Número máximo de fallos de nodo tolerados por un clúster Raft de 5 nodos ((5 - 1) / 2)) expresado en nodos.
Respuesta 2

Cálculos reproducidos

$((5 - 3) + 1) = 3$

C.7 Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js

Problema resuelto [PRB-0037]
Una aplicación SPA tradicional descarga un bundle monolítico de 480 KiB de JavaScript. Al migrar a una arquitectura híbrida SSR + Server Components, el 65 % del código de presentación y formateo permanece en el servidor. Calcule el tamaño en KiB del bundle JavaScript que aún debe descargar e hidratar el navegador.
Respuesta

$$Resultado = 168$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0039]
[CH-07.1] Calcule (Ahorro absoluto en KiB al reducir el bundle de 480 KiB a 168 KiB) expresado en KiB.
Respuesta 312
Ejercicio 2 [PRB-0040]
[CH-07.2] Calcule (Tiempo de parseo y compilación JS en móvil a razón de 1,5 ms por cada KiB para 168 KiB) expresado en ms.
Respuesta 252
Ejercicio 3 [PRB-0041]
[CH-07.3] Calcule (Nodos re-evaluados con Signals en una lista de 500 filas cuando cambia 1 celda frente a 500 sin memoización (ahorro)) expresado en nodos.
Respuesta 499
Ejercicio 4 [PRB-0042]
[CH-07.4] Calcule (Número de peticiones de red tras deduplicar 6 componentes que solicitan simultáneamente el mismo endpoint) expresado en peticiones.
Respuesta 1

Cálculos reproducidos

$\frac{(480\cdot(100-65))}{100} = 168$

C.8 Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter

Problema resuelto [PRB-0043]
Una estación de trabajo ejecuta simultáneamente 4 utilidades corporativas de escritorio. Empaquetadas con Electron, cada instancia consume 220 MB de memoria RAM debido a procesos Chromium independientes; migradas a Tauri (Rust + WebView compartido), cada instancia consume 45 MB. Calcule la memoria RAM total ahorrada en megabytes.
Respuesta

$$Resultado = 700$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
[CH-08.1] Calcule (Reducción del tamaño de descarga en MB al pasar de un instalador Electron de 115 MB a uno Tauri de 8 MB) expresado en MB.
Respuesta 107
Ejercicio 2 [PRB-0046]
[CH-08.2] Calcule (Presupuesto de tiempo por fotograma en milisegundos (redondeado a centésimas x100) para una app Flutter a 120 Hz (100000 / 125)) expresado en centésimas de ms.
Respuesta 800
Ejercicio 3 [PRB-0047]
[CH-08.3] Calcule (Capacidad total de peticiones por segundo en un clúster de 6 pods Axum que procesan 35 000 req/s cada uno) expresado en req/s.
Respuesta 210000
Ejercicio 4 [PRB-0048]
[CH-08.4] Calcule (Instancias de un servicio Scoped creadas durante un lote de 250 peticiones HTTP si se inyecta 2 veces por petición) expresado en instancias.
Respuesta 250

Cálculos reproducidos

$(4 \cdot (220 - 45)) = 700$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 28 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.

¶1: dos testimonios.

¶2: dos testimonios.

¶3: dos testimonios.

¶4: dos testimonios.

¶5: dos testimonios.

¶6: dos testimonios.

¶7: dos testimonios.

¶8: dos testimonios. Ecuaciones: 2 respaldada por una fuente.

¶9: dos testimonios.

¶10: dos testimonios.

¶11: dos testimonios.

¶12: dos testimonios.

¶13: dos testimonios.

¶14: dos testimonios.

¶15: dos testimonios.

¶16: dos testimonios.

¶17: dos testimonios.

¶18: dos testimonios.

¶19: dos testimonios.

¶20: dos testimonios. Ecuaciones: 1 respaldada por una fuente, 2 valor o condición.

¶21: dos testimonios.

¶22: dos testimonios.

¶23: dos testimonios.

¶24: dos testimonios.

¶25: dos testimonios.

¶26: dos testimonios.

¶27: dos testimonios.

¶28: dos testimonios.

¶29: dos testimonios.

¶30: dos testimonios.

¶31: dos testimonios.

¶32: dos testimonios.

¶33: dos testimonios.

¶34: dos testimonios.

¶35: dos testimonios.

¶36: dos testimonios.

¶37: dos testimonios.

¶38: dos testimonios.

¶39: dos testimonios.

¶40: dos testimonios.

¶41: dos testimonios.

¶42: dos testimonios.

¶43: dos testimonios.

¶44: dos testimonios.

¶45: dos testimonios.

¶46: dos testimonios.

¶47: dos testimonios.

¶48: dos testimonios.

HTML Living Standard & CSS Cascading and Inheritance Level 5 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Database System Concepts (7th Edition) & Designing Data-Intensive Applications (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Protocol Buffers v3, JSON Schema (RFC 8259) & YAML 1.2.2 Specification (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: ch-01 scientific
REV-0002: ch-02 scientific
REV-0003: ch-03 scientific
REV-0004: ch-04 scientific
REV-0005: ch-05 scientific
REV-0006: ch-06 scientific
REV-0007: ch-07 scientific
REV-0008: ch-08 scientific

Apéndice E

Plan de estudio

Partes VI–VIII: Web, Almacenamiento y Ecosistemas de Producción

✓HTML Living Standard y Accesibilidad Web (WCAG / ARIA)

- Dominar la semántica estructural de HTML5, el algoritmo de construcción del DOM, formularios nativos, multimedia y árboles de accesibilidad (AOM/ARIA).

2 conceptos planificados.

✓CSS Moderno: Cascada, Especificidad, Flexbox, Grid y Container Queries

- Calcular determinísticamente la resolución de la cascada CSS, diseñar layouts complejos con Flexbox y Grid, y aplicar propiedades lógicas y variables reactivas.

2 conceptos planificados.

✓Formatos de Intercambio, Configuración y Documentación: JSON, YAML, TOML, XML, CSV, Markdown y Protobuf

- Seleccionar y validar formatos de datos textuales y binarios, diseñar esquemas con JSON Schema y Protocol Buffers, y prevenir vulnerabilidades de deserialización.

2 conceptos planificados.

✓Teoría Relacional, Modelado Entidad-Relación y Normalización (1NF a BCNF)

- Traducir requisitos de dominio a esquemas relacionales en Forma Normal de Boyce-Codd (BCNF) garantizando reuniones sin pérdida (lossless-join) y preservación de dependencias.

2 conceptos planificados.

✓Motores de Almacenamiento, Índices B+-Tree / LSM, Transacciones ACID y MVCC

- Analizar la estructura física de páginas de disco, comparar árboles B+ frente a LSM-Trees, diseñar índices compuestos óptimos y depurar anomalías de concurrencia bajo niveles de aislamiento ANSI/serializable.

2 conceptos planificados.

✓Bases de Datos Distribuidas, NoSQL, Teorema CAP/PACELC, PostgreSQL, SQLite, MongoDB y Redis

- Diseñar topologías de replicación y particionamiento (sharding), evaluar compromisos CAP/PACELC y seleccionar con criterio técnico entre motores relacionales y NoSQL.

2 conceptos planificados.

✓Frameworks Web Frontend y Full-Stack: React, Angular, Vue, Svelte y Next.js

- Comprender los motores de reactividad (Virtual DOM, Signals, compilación estática), gestionar estado cliente/servidor e implementar Server-Side Rendering (SSR) e hidratación.

2 conceptos planificados.

✓Frameworks Backend, Escritorio y Móvil: FastAPI, Spring Boot, ASP.NET Core, Axum, Qt, Tauri y Flutter

- Diseñar tuberías de middleware backend, inyección de dependencias y validación de esquemas, y contrastar arquitecturas de escritorio y móvil nativas frente a multiplataforma.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] HTML Living Standard & CSS Cascading and Inheritance Level 5. https://html.spec.whatwg.org/vol-ref-1

[27] Database System Concepts (7th Edition) & Designing Data-Intensive Applications. https://www.db-book.com/vol-ref-2

[28] Protocol Buffers v3, JSON Schema (RFC 8259) & YAML 1.2.2 Specification. https://datatracker.ietf.org/doc/html/rfc8259/vol-ref-3